Game Stats & Recommendation API — API Documentation

Version: 1.0

Overview

- Base URL: depends on runtime host, e.g. `http://127.0.0.1:8000`.
- Authentication: JWT Bearer tokens. Include `Authorization: Bearer <access_token>` in requests that require authentication.

Error format

- Errors are returned as JSON, for example `{ "detail": "error message" }` or `{ "code": "ERR_CODE", "detail": "explanation" }`.
- Common HTTP status codes:
  - `200 OK` successful response

Auth

Scheme: HTTP Bearer JWT. Header: `Authorization: Bearer <access_token>`.

Example header:

```
Authorization: Bearer eyJhbGciOiJ...<snip>
```

POST /auth/register

- Body params (application/json): `username` (string, required), `email` (string, required), `password` (string, required)
- Success: `201 Created` with created user summary (id, username, email, role).

Example request:

```
curl -X POST http://127.0.0.1:8000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","email":"alice@example.com","password":"secret"}'
```

POST /auth/login

- Body params (application/json): `username` (string, required), `password` (string, required)
- Success: `200 OK` with JSON: `{ "access_token": "eyJ...", "token_type": "bearer" }`

Errors:

- `400` when request body invalid.

- `401` when credentials are incorrect.

GET /users/{user_id}/favorites

- Description: Get user's favorites (paginated). Implemented as `GET /users/{user_id}/favorites`.
- Query params: `skip` (default `0`), `limit` (default `50`). Response: array of `Favorite` objects, example:

```
[
  { "id": 7, "user_id": 1, "game_id": 2, "created_at": "2026-04-17T12:10:00Z" }
]
```

GET /users/{user_id}/reviews

- Description: Get user's reviews (paginated). Implemented as `GET /users/{user_id}/reviews`.
- Query params: `skip` (default `0`), `limit` (default `50`). Response: array of `Review` objects (see Reviews section for schema example).

Games

GET /games/

- Description: List games. Supported query params:
  - `q` search keywords (title/description)
  - `genre` filter by genre
  - `platform` filter by platform
  - `is_released` boolean
  - `min_rating` minimum average rating
  - `sort` e.g. `release_date`, `avg_rating`, `popularity` (append `:desc` for descending)
  - `skip` (integer, optional) — pagination offset, default `0`
  - `limit` (integer, optional) — page size, default `20`
- Notes: the implemented parameters are `skip` and `limit` (not `offset`), and `limit` defaults to 20. Use `skip` for pagination.
- Response example (paginated):

```
{
  "total": 12345,
  "items": [
    { "id": 1, "title": "My Game", "steam_appid": 123, "avg_rating": 4.2 },
    { "id": 2, "title": "Other Game", "steam_appid": 456, "avg_rating": 3.8 }
  ]
}
```

GET /games/{game_id}

- Description: Game details (includes aggregated rating and top reviews)
- Response example:

```
{
  "id": 1,
  "title": "My Game",
  "steam_appid": 123,
  "developers": ["Dev Studio"],
  "publishers": ["Pub Co"],
  "genres": ["Action"],
  "platforms": ["windows"],
  "release_date": "2024-11-12T00:00:00Z",
  "avg_rating": 4.2,
  "top_reviews": [ { "id": 10, "rating": 5, "content": "Great" } ]
}
```

POST /games/

- Description: Create a new game (publisher/dev/admin required)
- Request example:

```
{
  "title": "New Game",
  "steam_appid": 999999,
  "developers": ["Dev"],
  "publishers": ["Pub"],
  "genres": ["Indie"],
  "platforms": ["windows", "mac"],
  "description": "Short description",
  "release_date": "2026-06-01T00:00:00Z"
}
```

- Success: `201 Created` with created resource.
- Implementation note: the current codebase does NOT enforce authentication or publisher/admin checks for creating games. If you expect this to require auth, add `Depends(get_current_user)` and permission checks in the route implementation.

PATCH /games/{game_id}

- Description: Update game info (publisher or admin)
- Errors: `404 Not Found` if `game_id` does not exist; `401`/`403` for auth/permission issues.
- Implementation note: the current implementation does not enforce authentication/authorization for PATCH — callers can update game fields without a token. Consider adding auth enforcement if required.

DELETE /games/{game_id}

- Description: Delete or unpublish a game (publisher or admin), returns `204 No Content`.
- Implementation note: DELETE currently does not require authentication in the code. Ensure permission checks are implemented before relying on this in production.

Search

GET /search/

- Description: Full-text or fielded search. Params: `q`, `filters`, `limit`, `offset`.
- Example response:

```
[
  { "id": 1, "title": "Space Adventure", "steam_appid": 123, "avg_rating": 4.5 },
  { "id": 2, "title": "Space Miner", "steam_appid": 456, "avg_rating": 4.0 }
]
```

GET /search/suggestions

- Description: Search suggestions/autocomplete. Params: `q`, `limit`.
- Response example:

```
{ "suggestions": [ { "id": 123, "title": "Space Adventure" }, { "id": 456,
"title": "Space Miner" } ] }
```

Optional: GET /search/semantic (vector semantic search if embeddings/vector store integrated).

Reviews

GET /games/{game_id}/reviews

- Description: List reviews for a game (paginated, sortable)
- Example response (GET):

```
[
  { "id": 10, "user_id": 2, "game_id": 1, "rating": 5, "content": "Great!",
"created_at": "2026-04-17T12:00:00Z" },
  { "id": 11, "user_id": 3, "game_id": 1, "rating": 4, "content": "Pretty good",
"created_at": "2026-04-16T09:30:00Z" }
]
```

GET /reviews/games/{game_id}/reviews

- Description: List reviews for a game (paginated, sortable). Alias implemented as `GET /reviews/games/{game_id}/reviews`.

POST /reviews/games/{game_id}/reviews

- Description: Submit a review (auth required). Implemented under `/reviews/games/{game_id}/reviews`.
- Request example:

```
{
  "rating": 5,
  "content": "Really enjoyed this game!"
}
```

- Response (201 Created): review object, example:

```
{
  "id": 10,
  "user_id": 2,
  "game_id": 1,
  "rating": 5,
  "content": "Really enjoyed this game!",
  "created_at": "2026-04-17T12:00:00Z"
}
```

DELETE /reviews/{review_id}

- Description: Delete a review (author or admin). Implemented as `DELETE /reviews/{review_id}`.

POST /reviews/{review_id}/like

- Description: Like a review (auth required). Implemented as `POST /reviews/{review_id}/like`.
- Responses:
  - `201 Created` with body `{ "id": <like_id>, "review_id": <id> }` when a new like is created.
  - If user already liked the review, implementation returns `{ "detail": "already liked" }` (idempotent).

Favorites

POST /favorites/games/{game_id}/favorite

- Description: Add favorite (auth required), returns `201`.
- Response example (201 Created):

```
{
  "id": 5,
  "user_id": 2,
  "game_id": 1,
  "created_at": "2026-04-17T12:30:00Z"
}
```

DELETE /favorites/games/{game_id}/favorite

- Description: Remove favorite (auth required), returns `204`.

GET /favorites/users/me

- Description: Get current user's favorites (auth required, paginated). Implemented as `GET /favorites/users/me`. Use `Authorization: Bearer <access_token>`.
- Query params: `skip` (default 0), `limit` (default 50). Returns list of `Favorite` objects.
- Example response:

```
[
  { "id": 7, "user_id": 2, "game_id": 3, "created_at": "2026-04-17T12:10:00Z" }
]
```

Publisher / Admin features

GET /games/publishers/{publisher_id}/games

- Description: List games published by a publisher. Implemented as `GET /games/publishers/{publisher_id}/games` — returns games where the publisher is listed in the `publishers` JSON or matches the developer id.

Admin sample endpoints

- GET /admin/users
- PATCH /admin/users/{id}/role
- POST /admin/ban-user
- Note: the above admin endpoints are listed as examples; they are not implemented in the current codebase. Remove or implement before relying on them.

Media & Assets

POST /games/{game_id}/assets

- Description: Upload images/videos (auth/publisher required)
- Content-Type: `multipart/form-data`.
- Form fields:
  - `file` (file) — image or video (required)
  - `type` (string, optional) — `image` or `video`

Example (curl):

```
curl -X POST "http://127.0.0.1:8000/games/1/assets" \
  -H "Authorization: Bearer <token>" \
  -F "file=@screenshot.png" -F "type=image"
```

- Notes & Response: multiple files may be submitted (parameter `files: list[UploadFile]`). On success the endpoint returns a JSON array of created `GameAsset` objects; example:

```
[
  { "id": 3, "game_id": 1, "filename": "screenshot.png", "url":
"data/assets/1/screenshot.png" },
  { "id": 4, "game_id": 1, "filename": "screenshot2.png", "url":
"data/assets/1/screenshot2.png" }
]
```

Recommendations

GET /recommendations/personal

- Description: Personalized recommendations based on user behavior (auth required)
- Auth: requires `Authorization: Bearer <access_token>`.
- Query params: `limit` (default 20).
- Response example (array of Game objects):

```
[
  { "id": 12, "title": "Space Adventure", "steam_appid": 321, "avg_rating": 4.6 },
  { "id": 45, "title": "Indie Puzzle", "steam_appid": 654, "avg_rating": 4.3 }
]
```

GET /recommendations/popular

- Description: Return popular/trending recommendations
- Query params: `limit` (default 20). Response example: same format as personal recommendations.

Health & Docs

GET /health

- Description: Health check, returns `{ "status": "ok" }`.

GET /openapi.json

- Description: OpenAPI spec (auto-generated by FastAPI). Use this for programmatic clients, test generation, or to feed tools like Swagger Codegen.

GET /docs

- Description: Swagger UI (auto-provided by FastAPI). Interactive browser UI for manual testing: explore endpoints, fill parameters, send requests, and view responses. Note: `/docs` is a UI page and not part of `paths` in `openapi.json` — automated spec comparisons may list it as a doc-only entry.

Error Codes (expanded)

- 400 Bad Request
  - JSON body: `{ "detail": "..." }` or `{ "code": "ERR_CODE", "detail": "..." }`
  - Example: `{ "detail": "Invalid value for 'rating', must be 1-5" }`
- 401 Unauthorized
  - Example: `{ "detail": "Not authenticated" }`
- 403 Forbidden
  - Example: `{ "detail": "Insufficient permissions" }`
- 404 Not Found
  - Example: `{ "detail": "Game not found" }`
- 429 Too Many Requests
  - Example: `{ "detail": "Rate limit exceeded" }`
- 500 Internal Server Error
  - Example: `{ "detail": "Internal server error" }`

-

Seed script

- Description: Import initial game data and optional sample users/favorites/reviews from a CSV file. The repository includes `scripts/seed.py` which by default reads `data/steam_games.csv`.

Usage (local):

```
python scripts/seed.py --file data/steam_games.csv --limit 200 --skip-existing
```

- The script creates hashed passwords for sample users, skips duplicates by default when `--skip-existing` is provided, and prints a short summary after import.

Convert to PDF

- We recommend using `pandoc` or printing to PDF from an editor (VS Code).

```
pandoc docs/API.md -o docs/API.pdf --from markdown
```

Windows PowerShell example:

```
pandoc docs\API.md -o docs\API.pdf
```

Maintenance notes

- Add the generated `docs/API.pdf` to the repository and reference it in `README.md`.